```
import cv2
import numpy as np

video_path = "/content/Screen Recording 2026-06-11 120518.mp4"


def lucas_kanade_tracking():
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 20
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    out = cv2.VideoWriter("output_lucas_kanade.mp4", fourcc, fps, (width, height))

    feature_params = dict(maxCorners=100, qualityLevel=0.3, minDistance=7, blockSize=7
    lk_params = dict(winSize=(15, 15), maxLevel=2,
                     criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0
    color = np.random.randint(0, 255, (100, 3))

    ret, old_frame = cap.read()
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
    p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
    mask = np.zeros_like(old_frame)

    frame_count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        if p0 is None or len(p0) == 0:
            p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
            mask = np.zeros_like(frame)
            old_gray = frame_gray.copy()
            out.write(frame)
            frame_count += 1
            continue

        p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_pa

        if p1 is None:
            p0 = cv2.goodFeaturesToTrack(frame_gray, mask=None, **feature_params)
            mask = np.zeros_like(frame)
            old_gray = frame_gray.copy()
            out.write(frame)
            frame_count += 1
            continue

        good_new = p1[st == 1]
        good_old = p0[st == 1]
```

```
            for i, (new, old) in enumerate(zip(good_new, good_old)):
                a, b = new.ravel()
                c, d = old.ravel()
                mask = cv2.line(mask, (int(a), int(b)), (int(c), int(d)), color[i].tolist(
                frame = cv2.circle(frame, (int(a), int(b)), 5, color[i].tolist(), -1)

            img = cv2.add(frame, mask)
            out.write(img)

            old_gray = frame_gray.copy()
            p0 = good_new.reshape(-1, 1, 2)

            if len(good_new) < 5:
                p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)
                mask = np.zeros_like(frame)

            frame_count += 1

        cap.release()
        out.release()
        print("Frames processed:", frame_count)
        print("Saved: output_lucas_kanade.mp4")


    def farneback_tracking():
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) or 20
        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

        fourcc = cv2.VideoWriter_fourcc(*"mp4v")
        out = cv2.VideoWriter("output_farneback.mp4", fourcc, fps, (width, height))

        ret, frame1 = cap.read()
        prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
        hsv = np.zeros_like(frame1)
        hsv[..., 1] = 255

        while True:
            ret, frame2 = cap.read()
            if not ret:
                break

            next_frame = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
            flow = cv2.calcOpticalFlowFarneback(prvs, next_frame, None, 0.5, 3, 15, 3, 5,

            mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
            hsv[..., 0] = ang * 180 / np.pi / 2
            hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
            bgr = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

            out.write(bgr)
            prvs = next_frame

        cap.release()
        out.release()
        print("Saved: output_farneback.mp4")
```

```
if __name__ == "__main__":
    print("1. Lucas-Kanade Sparse Optical Flow")
    print("2. Farneback Dense Optical Flow")
    choice = input("Enter choice (1/2): ")

    if choice == "1":
        lucas_kanade_tracking()
    elif choice == "2":
        farneback_tracking()
    else:
        print("Invalid choice")
```

```
1. Lucas-Kanade Sparse Optical Flow
2. Farneback Dense Optical Flow
Enter choice (1/2): 1
Frames processed: 424
Saved: output_lucas_kanade.mp4
```

PART - B

1. Object tracking vs. object detection Detection finds an object in one frame ("there's a person here"). Tracking follows that same object across many frames over time ("this person moved from left to right").
2. How Optical Flow helps real-time tracking It measures how pixels move between two frames right after each other. By following these small movements frame by frame, the system knows where the object went — without needing to re-detect it every single time.
3. Role of Shi-Tomasi in Lucas-Kanade Shi-Tomasi picks the best points to track — corners and sharp edges that are easy to follow reliably. Lucas-Kanade then tracks how these chosen points move. Good starting points = more accurate tracking.
4. Why Optical Flow suits motion analysis It directly reads pixel brightness changes to calculate speed and direction of movement — smooth, fast, and doesn't need a trained model, making it great for analyzing motion patterns.
5. Challenges: fast or hidden objects

Fast objects: move too far between frames, so the algorithm can lose them. Occluded (partly hidden) objects: feature points disappear, breaking the tracking chain. Lighting changes and camera shake also confuse tracking.

6. Optical Flow vs. deep learning tracking Optical Flow: fast, light, no training needed — but struggles with occlusion, lighting, sudden motion. Deep learning trackers: better at handling occlusion and appearance changes, can "re-find" lost objects — but need more computing power and training data.
7. Use in traffic & self-driving systems Traffic monitoring: measures vehicle speed, counts cars, spots wrong-way driving. Self-driving cars: understands how nearby vehicles/pedestrians move, helping avoid collisions and plan safe paths.

8. Effect of camera motion If the camera itself shakes or pans, even the still background looks like it's moving. This can trick the system into thinking there's object motion when there isn't — so camera motion needs to be separated out first.
9. Five real-world applications

Traffic monitoring Sports analytics (tracking players/ball) Video surveillance Robotics & drone navigation Human activity recognition

10. Improving surveillance, robotics & activity recognition

Surveillance: spots unusual motion (intrusion, abandoned bags). Robotics: helps the robot sense its own movement and avoid obstacles. Activity recognition: picks up motion patterns like walking, running, or falling — even subtle ones static images would miss.

```
...
```

```
Ellipsis
```

```
Start coding or generate with AI.
```